

CreditTech — Data Sourcing, Training & Modeling Specification

Execution-Level Plan for the ML/Data Workstream: What Data, From Where, How Trained, With What Parameters

Purpose: this document is the detailed engineering specification for CreditTech's data and modeling workstream — precise enough that someone unfamiliar with the project could acquire the same data, build the same feature set, train the same models with the same parameters, and evaluate them the same way. It defines **method, not outcome**: since no real pilot repayment data exists yet, it contains no trained-model results or performance numbers. Those will be produced and presented once real data is collected (trigger conditions in §7 and §11).

Scope	Dataset sourcing → labeling → feature engineering → preprocessing → training → tuning → promotion gate → evaluation → explainability → reproducibility
Out of scope here	Trained-model metrics/results (none exist pre-pilot); system architecture, deployment, and pilot logistics — see the companion Mid-Term Conceptual Design Plan
Status legend	REAL/SANDBOX data reachable today via a registered API PUBLIC open government/research data SYNTHETIC/PROXY generated or borrowed for pipeline validation only

1. Dataset Catalog — Exactly Where Every Input Comes From

Five acquisition rails feed the feature table. Each is independently reachable/degradable (§2), so a missing rail lowers scoring confidence rather than blocking it.

1.1 Rail A — Account Aggregator (financial / transaction / digital-activity data)

Item	Detail
What it supplies	Bank balance history, credit inflows, UPI transaction count/volume, account age, utility-bill payment timeliness
Source & access	REAL/SANDBOX — a licensed NBFC-Account Aggregator under the Sahamati/RBI AA ecosystem (e.g. Setu AA, Finvu, CAMSFinServ sandbox environments for development). Registration as a Financial Information User (FIU) is required; sandbox <code>client_id</code> / <code>client_secret</code> credentials are issued after onboarding.
Acquisition protocol	OAuth-style consent handshake → AA gateway issues a consent artifact (purpose-scoped, time-bound) → FIU pulls Financial Information (FI) in the AA-specified JSON schema (<code>FI-Notification</code> → <code>FI-Fetch</code>) → response parsed into raw transaction/account records
License / cost	Per-consent fee, scales with applications (see cost driver §16 of the companion plan); sandbox tier is free for development
Refresh cadence	On-demand per application at origination; re-pulled during post-disbursement monitoring (planned, post-pilot)

1.2 Rail B — Geospatial / Earth Observation (agricultural / farm / weather data)

Item	Detail
What it supplies	NDVI (crop health), NDVI trend across seasons, rainfall deviation, irrigation-access proxy
Primary source	REAL/SANDBOX — contracted commercial geospatial vendor (SLA-backed; contract still open — Phase-0 external item E3)
Interim/fallback source	PUBLIC — Sentinel-2 Level-2A surface reflectance via Copernicus Open Access Hub or Google Earth Engine (<code>COPERNICUS/S2_SR_HARMONIZED</code> collection), free tier, ~10 m resolution, ~5-day revisit
Acquisition protocol	Register a Google Earth Engine account → define the borrower's farm polygon (or a buffer around the geocoded plot centroid where a polygon isn't available) → filter the Sentinel-2 collection by date range + cloud-cover mask → compute NDVI per pixel → aggregate (mean) over the polygon → repeat for the prior season to get the 2-season trend
Core formula	$NDVI = (NIR - RED) / (NIR + RED)$ using Sentinel-2 bands B8 (NIR) and B4 (RED)
Rainfall source	PUBLIC — India Meteorological Department (IMD) gridded daily rainfall (0.25°×0.25°), compared against the borrower's block/district long-period average to compute deviation %

1.3 Rail C — Bureau (Credit Information Company)

Item	Detail
What it supplies	Existing formal loan status, KCC holding/utilization, prior bureau score where one exists
Source	REAL/SANDBOX — a Credit Information Company (CIC) test/sandbox environment (e.g. CIBIL, Experian, Equifax, CRIF High Mark developer sandboxes)
Acquisition protocol	API key issued after commercial agreement → request by Aadhaar-reference hash (never raw Aadhaar) → XML/JSON credit report parsed for the specific fields the feature table needs, all other fields discarded at ingestion (data minimisation)
Expected coverage	Low — most pilot borrowers are thin-file/no-file by definition (§1.1 of the companion plan); this rail is expected to return "no record" often, which is itself informative and preserved as a missingness indicator (§5)

1.4 Rail D — SHG / FPO Manual Entry (group-level behavioural data)

Item	Detail
What it supplies	SHG repayment rate, meeting attendance, savings consistency, membership years, NABARD SHG grade
Source	REAL (field) — physical SHG passbooks/registers, digitised by a Bank Sakhi through an offline-first entry UI; PUBLIC aggregate reference — NABARD's SHG-Bank Linkage Programme (SHG-BLP) published statistics, used to sanity-check field entries, not as a per-borrower source
Acquisition protocol	Bank Sakhi enters data on a mobile device (works offline) → dual-entry reconciliation (two independent entries compared, discrepancies flagged for manual review) → batch-synced (SFTP or scheduled API import) when connectivity is available

1.5 Rail E — Government / Public Program Data

Item	Detail
What it supplies	PM-KISAN beneficiary status & transfer regularity, PMFBY crop-insurance enrolment, KCC statistics
Source	PUBLIC — PM-KISAN beneficiary portal, PMFBY portal, data.gov.in open API (registration required, free API key), Agmarknet / e-NAM daily mandi price bulletins
Acquisition protocol	data.gov.in: register for an API key → query resource-specific REST endpoints (JSON/CSV) → scheduled daily/weekly pull for mandi prices; PM-KISAN/PMFBY: either direct portal query by beneficiary ID (where borrower consents to share it) or manual entry cross-check by Bank Sakhi

1.6 Rail F — Public/Research Datasets Used for Modeling-Technique Validation Only

Important scope boundary: the datasets below are **not agricultural and not CreditTech's own borrowers**. They exist purely to validate that the modeling pipeline (encoding, training, evaluation, fairness tooling) works correctly on labelled data before real pilot outcomes exist. Their results must never be presented as CreditTech's own performance.

Dataset	Access	Why used
Kaggle — <i>Home Credit Default Risk</i>	kaggle competitions download -c home-credit-default-risk	Closest public analog: thin-file/alternative-data-style features, real default labels, ~300 features — best pipeline-validation proxy
UCI / Kaggle — <i>German Credit Data (Statlog)</i>	kaggle datasets download -d uciml/german-credit	Classic small scorecard benchmark; good for unit-testing the WoE/IV and logistic-scorecard code path cheaply
Kaggle — <i>Give Me Some Credit</i>	kaggle competitions download -c GiveMeSomeCredit	Behavioural/utilisation features analogous to the Capacity/Capital feature groups

1.7 Synthetic Data (already scaffolded in the repository)

Component	Role
services/core/ingestion/mocks/server.py	In-process mock API server returning synthetic AA/geospatial/bureau payloads — lets the ingestion orchestrator and feature pipeline be built, tested, and demoed before real vendor credentials exist
simulate_pilot.py	Synthesises a full borrower cohort (all five rails) to exercise consent → ingestion → scoring → decision end-to-end for demos, CI, and load testing

2. Step-by-Step Data Acquisition Procedure

```
For each application:
1. Borrower gives Aadhaar e-KYC + purpose-scoped consent --> consent hash-chained & stored
2. Orchestrator fires all reachable rails IN PARALLEL (asyncio.gather), each with its own timeout:
  AA connector      : timeout 10s, retry x3, exponential backoff (2s,4s,8s)
  Geospatial conn. : timeout 15s, no retry (idempotent re-query next cycle if needed)
  Bureau connector  : timeout 8s,  retry x3, exponential backoff
  SHG/FPO          : pre-loaded from last Bank Sakhi sync (no live call at scoring time)
3. Each rail returns DATA, TIMEOUT, or ERROR -- never blocks the others (graceful degradation)
4. Raw payloads validated against a Pydantic schema per rail -- malformed records rejected + logged
5. Validated raw records written to rail-specific staging tables, tagged with a pull_id + timestamp
6. Feature Engineering stage (Sec 4) consumes staging tables, never raw payloads directly
```

This mirrors the orchestrator already scaffolded in `services/core/ingestion/service.py` and `connectors.py` (parallel fetch, per-rail timeout, tenacity-based retry) — the plan above is the same design, made explicit for the ML/data workstream.

3. Target-Variable / Labeling Plan

Decision	Specification
Primary default proxy	DPD90+ — 90 or more days past due on a scheduled repayment, measured on the partner RE's loan-management ledger
SHG-routed loan proxy	Where individual DPD tracking doesn't apply cleanly, use missed repayment across a full crop cycle (Kharif or Rabi) as the default event
Observation window	Outcome is only labelled once the loan has had at least one full scheduled-repayment cycle to mature; loans still within their first cycle are treated as <i>censored</i> , not "good," and excluded from training until resolved
Minimum data volume to begin training	≥ 200 resolved outcomes (default + non-default combined) after ≥ 6 months of pilot operation — below this, sampling noise makes any trained model unreliable; the deterministic scorecard remains the only production model
Known proxy risk (documented, not hidden)	DPD90+ can misclassify hardship-driven late payment (e.g. a failed monsoon) as equivalent to unwillingness to repay. This is a real fairness/regulatory-alignment risk in an agricultural portfolio and must be reviewed alongside the fairness gate (§7), not treated as a purely technical labeling detail

4. Full Feature Catalogue & Calculation Logic

The feature set is organised under the "5 Cs" framework. Below is the calculation logic for a representative feature in each category (the complete 40+ feature table lives in `docs/phase0/preliminary_feature_list.md`; this section shows the calculation method, which that file does not spell out).

Feature ID	Name	Calculation	Rail
C001	<code>shg_repayment_rate</code>	<code>on_time_repayments / total_scheduled_repayments</code> over trailing 12 months	SHG/FPO
C008	<code>upi_transaction_regularity</code>	Coefficient of variation: <code>std(monthly_txn_count) / mean(monthly_txn_count)</code> — lower = more regular	AA
CA001/2	<code>estimated_crop_income_kharif/rabi</code>	NDVI-derived yield proxy × cultivated area (acres) × block-level mandi price (Agmarknet)	Geo + Public
CA003	<code>income_stability_cv</code>	<code>std(monthly_income) / mean(monthly_income)</code> over trailing 12 months	AA/Manual
CP002/3	<code>bank_balance_avg_6m / min_6m</code>	Mean / minimum of end-of-day balance over trailing 6 months	AA
CP005/6	<code>kcc_holder / kcc_utilization_pct</code>	Binary flag; <code>outstanding_kcc_balance / kcc_sanctioned_limit × 100</code>	Bureau
CL004	<code>land_quality_ndvi_avg</code>	Mean NDVI over the farm polygon, averaged across the last 2 seasons	Geo
CO001	<code>ndvi_trend_2season</code>	<code>NDVI(season_t) - NDVI(season_t-1)</code> , normalised to [-1,1]	Geo
CO002	<code>rainfall_deviation_pct</code>	<code>(actual_rainfall - block_LPA) / block_LPA × 100</code> (LPA = long-period average, IMD)	Public
D001/2	<code>upi_monthly_txn_count / volume</code>	Mean monthly count/sum over trailing 6 months from AA transaction ledger	AA

4.1 Reference/baseline values used for SHAP-style contributions

Each feature has a cohort reference value (the rural-borrower-cohort mean/typical value) against which a given applicant's contribution is measured — e.g. `shg_repayment_rate` reference = 90.0, `bank_balance_avg_6m` reference = ₹5,000. These references are already defined in `ml/evaluation/explain.py` and must be recalibrated once real pilot-cohort statistics exist (§9).

5. Preprocessing Specification

Raw staging tables --> Schema validation (Pydantic) --> Cleaning (unit/label normalisation)
--> Missing-value treatment --> Outlier treatment --> WoE encoding --> Scaling (baseline model only)
--> Feature construction (derived ratios) --> Dataset validation (assertions) --> Split

Step	Exact method
Missing-value treatment	Per-feature missingness indicator column preserved (never silently imputed away) + median (continuous) / mode (categorical) imputation <i>within segment</i> (Farmer/SHG/FPO/Enterprise) — a global mean would blur genuinely different income shapes across segments
Outlier treatment	Winsorise continuous features at the 1st/99th percentile within segment; domain-sanity hard bounds (e.g. <code>land_holding_acres > 50</code> flagged for manual review, never auto-dropped)
Categorical encoding — candidate model	Weight-of-Evidence (WoE): <code>WoE_bin = ln(%non-default_in_bin / %default_in_bin)</code> , computed per bin, fit on the training fold only and applied unchanged to validation/test (leakage prevention)
Feature selection filter	Information Value (IV): <code>IV = Σ (%non-default - %default) × WoE</code> per feature. Standard convention applied: <code>IV < 0.02</code> dropped (not predictive); <code>0.02–0.1</code> weak (kept, monitored); <code>0.1–0.3</code> medium; <code>0.3–0.5</code> strong; <code>> 0.5</code> flagged as suspicious and manually reviewed for leakage before use
Scaling	Standardisation (zero mean, unit variance) for the logistic baseline's continuous inputs only; tree-based candidates (XGBoost/LightGBM/CatBoost) use raw/WoE-encoded values — scaling is not required for tree splits
Categorical handling — CatBoost path	Native categorical support for <code>crop_type_primary</code> , <code>land_ownership_type</code> , <code>shg_grade</code> , <code>site_type</code> — no encoding needed if CatBoost is selected (§6)
Train / validation / test split	Stratified by target label AND grouped by <code>village_id</code> (StratifiedGroupKFold) so no village's borrowers appear in both train and validation — prevents village-level leakage; nominal split 70/15/15; time-aware — validation/test rows must come from a resolution date after the training rows' resolution date, to respect the crop-cycle sequence

6. Model Training Plan

6.1 Baseline — Logistic Scorecard (already scaffolded; deployed at pilot launch)

This is **not a trained model** pre-pilot — it is a deterministic, hand-set weighted logistic function calibrated from published "5 Cs" literature weighting (Jonnalagadda & Babu, 2026), already implemented in `ml/training/scorecard.py`. Its exact form:

```
log_odds = intercept (-1.2) + Σ (weight_i × feature_value_i)
prob_repayment = 1 / (1 + exp(-log_odds))
score_100 = round(prob_repayment * 100, 1)
score_900 = offset + factor * ln( prob_repayment / (1 - prob_repayment) )
    where factor = 20 / ln(2), offset = 600 - factor * ln(50) # PD0=20, 50:1 odds at 600
```

Recalibration procedure once real outcomes exist: refit intercept + weights via maximum-likelihood logistic regression (`sklearn.linear_model.LogisticRegression`, `penalty='l2'`, regularisation strength `C` tuned by grid `[0.01, 0.1, 1, 10]` via 5-fold CV on AUC) on the WoE-encoded feature set, keeping the same feature list for continuity of the reason-code templates.

6.2 Candidate Models — Trained Once Real Outcomes Exist (post-pilot trigger, §3)

Model	Hyperparameter search space	Notes
XGBoost	<code>max_depth</code> : [3,4,5,6,8] · <code>learning_rate</code> : [0.01,0.05,0.1] · <code>n_estimators</code> : up to 500 with <code>early_stopping_rounds=50</code> · <code>subsample</code> : [0.7,0.8,0.9,1.0] · <code>colsample_bytree</code> : [0.6,0.8,1.0] · <code>min_child_weight</code> : [1,3,5,10] · <code>reg_alpha</code> : [0,0.1,1] · <code>reg_lambda</code> : [1,5,10] · <code>scale_pos_weight</code> = neg/pos ratio	Primary candidate; <code>objective='binary:logistic'</code> , <code>eval_metric=['auc','aucpr']</code>
LightGBM	<code>num_leaves</code> : [15,31,63] · <code>learning_rate</code> : [0.01,0.05,0.1] · <code>feature_fraction</code> : [0.6,0.8,1.0] · <code>bagging_fraction</code> : [0.7,0.9] · <code>min_child_samples</code> : [10,20,50] · <code>lambda_l1/l2</code> : [0,1,5] · <code>is_unbalance=True</code>	Alternative candidate — faster on larger pilot-scale datasets
CatBoost	<code>depth</code> : [4,6,8] · <code>learning_rate</code> : [0.01,0.05,0.1] · <code>l2_leaf_reg</code> : [1,3,5,9] · <code>iterations</code> : up to 500 with	Evaluate only if XGBoost/LightGBM underperform on categorical-heavy segments (SHG grade, crop type)

6.3 Search strategy, imbalance handling, and reproducibility

Setting	Value / method
Hyperparameter search	Bayesian search via Optuna, 100 trials per candidate model, optimising validation PR-AUC (primary, due to class imbalance) with AUC and Expected Calibration Error tracked as secondary constraints
Cross-validation	5-fold StratifiedGroupKFold (grouped by <code>village_id</code> , stratified by target) — same scheme as the train/val/test split (§5)
Class-imbalance handling	Primary: <code>scale_pos_weight</code> (XGBoost) / <code>is_unbalance</code> (LightGBM) / <code>class_weights</code> (CatBoost) — cost-sensitive learning rather than resampling. SMOTE-style synthetic oversampling is explicitly not the default (flagged as a leakage risk on grouped/temporal folds) and would only be considered with strict fold-boundary isolation
Early stopping	50 rounds without validation-metric improvement, on the held-out fold within each CV split
Random seed policy	<code>random_state=42</code> fixed across every split, model, and search process — pinned in the shared training config, not scattered per script
Post-hoc calibration	Isotonic regression (non-parametric, preferred when enough resolved outcomes exist — pilot volume is modest) or Platt scaling (parametric fallback if sample size per bin is too small for isotonic) applied on a held-out calibration fold, evaluated via reliability diagrams + Brier score (§8)
Compute / environment	CPU-only training (tabular gradient-boosted trees on hundreds–low-thousands of rows do not benefit from GPU); library versions pinned via <code>pyproject.toml</code> 's <code>ml_extra</code> (scikit-learn, xgboost, lightgbm, shap, pandas, numpy)

7. Model Selection & Promotion Gate

A candidate model is promoted to replace the baseline as the production scorer **only if every condition below holds** — beating baseline accuracy alone is explicitly insufficient.

Gate condition	Threshold
Discrimination improvement	Candidate AUC $\geq$ Baseline AUC + 0.03, statistically significant (DeLong test, $p < 0.05$)
Rank-ordering strength	KS statistic ≥ 0.30
Calibration quality	Brier score $\leq$ Baseline Brier score; Expected Calibration Error ≤ 0.05
Fairness — demographic parity	Approval-rate gap across gender / landholding-band / geography ≤ 5 percentage points
Fairness — equalised odds	True-positive-rate gap across the same dimensions ≤ 7 percentage points
Minimum evaluable sample	≥ 200 resolved outcomes overall, and ≥ 30 per segment being separately evaluated — a segment below this is reported as "insufficient data," never silently passed
Human sign-off	Promotion is human-approved after the automated gate passes — never a fully automatic deploy

If a candidate fails any condition, the deterministic baseline remains in production and the candidate's evaluation report is logged for the next retraining cycle — a failed candidate is diagnostic information, not a dead end.

8. Evaluation Protocol

Metric	Computed how	Why it's included
ROC-AUC / PR-AUC	<code>sklearn.metrics.roc_auc_score</code> / <code>average_precision_score</code> on held-out fold	PR-AUC specifically compensates for class imbalance, where ROC-AUC alone can look optimistic
KS statistic	Max separation between cumulative good/bad distributions across score deciles	Standard credit-scorecard rank-ordering metric, familiar to lending stakeholders
Precision / Recall / F1	At the selected operating threshold (§8.1)	Names the false-positive/false-negative trade-off explicitly rather than hiding it in one number
Calibration curve + Brier score	10-bin reliability diagram; <code>sklearn.metrics.brier_score_loss</code>	The score is presented to loan officers as a probability-derived number — it must mean what it says
Confusion matrix	At selected threshold, per segment	Makes the FP/FN cost trade-off visible per Farmer/SHG/FPO/Enterprise segment

Segment-wise re-computation	Every metric above, recomputed separately per borrower segment and per pilot site	An aggregate-only metric can hide poor performance on a specific segment or site
Fairness metrics	Demographic parity gap, equalised-odds gap (§7)	Mandatory pre-promotion gate, not a post-hoc report

8.1 Threshold selection

The operating threshold (score cut-off separating Low/Medium/High risk categories) is chosen against the partner RE's stated cost ratio of a missed default vs. a wrongly-denied applicant — not fixed at the default 0.5 probability cut-off — using a cost-weighted threshold sweep over the validation fold.

9. Explainability Pipeline Specification

Stage	Method
Baseline (linear) model	Exact Shapley decomposition: <code>SHAP_i = coefficient_i × (value_i - reference_i)</code> — already implemented in <code>ml/evaluation/explain.py</code> ; exact because the model is additive/linear
Candidate (tree) model	<code>shap.TreeExplainer</code> from the open-source <code>shap</code> library — required once the model is non-linear, since the linear closed-form no longer applies. Background sample: 200 randomly sampled training rows (interventional feature perturbation)
Reason-code selection	Top-5 features by absolute SHAP value per application, mapped to the bilingual (English/Hindi) template library (<code>docs/phase0/shap_reason_code_templates.md</code>)
Global explanation	Mean absolute SHAP value per feature across the validation set, published alongside each model version in the registry

10. Reproducibility & Versioning

Element	Policy
Random seeds	Single fixed <code>random_state=42</code> , set once in a shared config, not per-script
Library versions	Pinned in <code>pyproject.toml</code> 's <code>ml_extra</code> (scikit-learn, xgboost, lightgbm, shap, pandas, numpy) — exact versions locked, not floating ranges, for training code
Data versioning	DVC (or equivalent) pointed at <code>ml/registry/</code> once real training data exists — every training run references an immutable data-snapshot hash
Model registry entry	Model artifact + hyperparameters + data-snapshot reference + full evaluation report (§7, §8), versioned and never overwritten
Config-driven training	All of §6's hyperparameter grids and thresholds live in a single versioned YAML config (example below), not hard-coded in scripts

```
model: xgboost
seed: 42
cv:
  scheme: stratified_group_kfold
  n_splits: 5
  group_col: village_id
search:
  method: optuna
  n_trials: 100
  objective_metric: pr_auc
  space:
    max_depth: [3,4,5,6,8]
    learning_rate: [0.01,0.05,0.1]
    n_estimators: {max: 500, early_stopping_rounds: 50}
    subsample: [0.7,0.8,0.9,1.0]
    colsample_bytree: [0.6,0.8,1.0]
promotion_gate:
  min_auc_gain: 0.03
  min_ks: 0.30
  max_ece: 0.05
  max_fairness_gap_pp: 5
  min_resolved_outcomes: 200
```

11. What This Document Deliberately Excludes, and What Happens Next

No trained-model results, accuracy numbers, or score distributions appear in this document — none exist yet. The V1 production model is the deterministic scorecard (§6.1), not a trained classifier; candidate-model training (§6.2) has an explicit data-

volume trigger (§3) that has not yet been reached. Presenting invented numbers here would misrepresent the project's actual state.

What happens	Trigger condition
Baseline scorecard goes live	Pilot launch (site 1), immediately — no training data required (§6.1)
Baseline recalibration (refit weights on real data)	First batch of resolved outcomes, even before the 200-case candidate threshold
Candidate model training begins	≥ 200 resolved outcomes AND ≥ 6 months of pilot operation (§3)
Candidate evaluation report produced	Immediately after training — presented regardless of whether the promotion gate (§7) passes
Results/metrics presented	Once the above exist — this is the natural "final result" section for a later submission, deliberately not fabricated here

12. Assumptions, Risks & Open Questions (ML/Data Workstream)

Item	Note
AA/geospatial/bureau sandbox access	Assumed obtainable during the build phase even where production vendor contracts (Phase-0 external items) are still pending — sandbox tiers are typically free/low-friction
Sentinel-2 fallback resolution	10 m/~5-day revisit is coarser than a commercial vendor's SLA — acceptable for pilot validation, not asserted as production-equivalent
Proxy-label risk (DPD90+)	Documented explicitly in §3 — a known, not hidden, source of potential misclassification in an agricultural portfolio
Public benchmark datasets (§1.6)	Used only to validate pipeline mechanics — never conflated with CreditTech's own population or presented as pilot results
Sample size at pilot scale	Hundreds, not thousands, of resolved outcomes in year one — this is precisely why the interpretable baseline remains the production fallback even after a candidate is trained (§6.1, §7)
Single-vendor geospatial dependency	Mitigated by the open-data fallback (§1.2) but not eliminated — tracked as Phase-0 external item E3 in the companion roadmap

Companion documents: [CreditTech_MidTerm_Conceptual_Design_Plan.pdf](#) (architecture, deployment, pilot design, cost, roadmap) · [Credittechplanningcontext.md](#) (full engineering roadmap) · [docs/phase0/preliminary_feature_list.md](#) (complete 40+ feature table) · [docs/research/external_benchmark_analysis.md](#) (comparative review informing several choices above).